# RELATÓRIO DE LABORATÓRIO

# Nº2 MICROPROCESSADORES

# CONTROLO DE PH E FILTRAGEM DE ÁGUA USANDO UM ARDUINO

Aluno: Alexandre Saraiva
nº44485 EEC
Docente: Prof. Dr. António Pinheiro
Disciplina: Microprocessadores
Covilhã
2024

# 1. INDICE

1. Introdução……1

2. Problema……..2

3.Código em Assembly……3

4. Tabela de teste…….,,,,,……4

5. Conclusão……5

# 2. INTRODUÇÃO

Este trabalho tem como finalidade a implementação de um sistema de Controlo, que consiste em controlar um sistema de filtragem de água de uma piscina e regular o seu Ph usando um Arduíno ATmega 2560.
A implementação do sistema de controlo vai exigir a montagem do circuito em hardware e a elaboração de um programa na linguagem Assembly em código específico para o ATmega 2560.
Para começar vou abordar o problema descrevendo os passos a seguir e elaborar um esquema representativo.

# 3. PROBLEMA

Pretende-se controlar um sistema de filtragem de água de uma piscina usando um arduino.
O sistema tem de ativar o motor de filtragem de água quando um relógio e o indica.
Além disso durante essa operação, deve usar o sistema de correção automático para o sistema de Ph caso ele não seja o adequado.

Sendo assim, o sistema deve funcionar da seguinte forma:

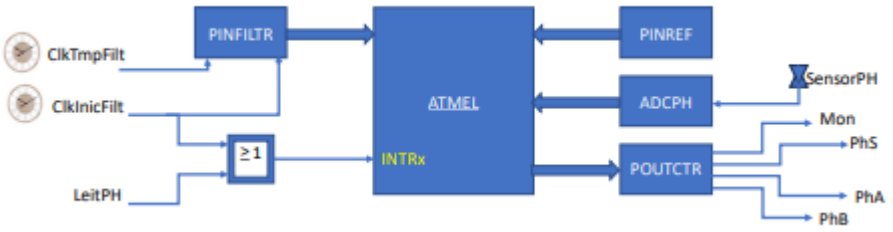
1) Tem um porto de controlo

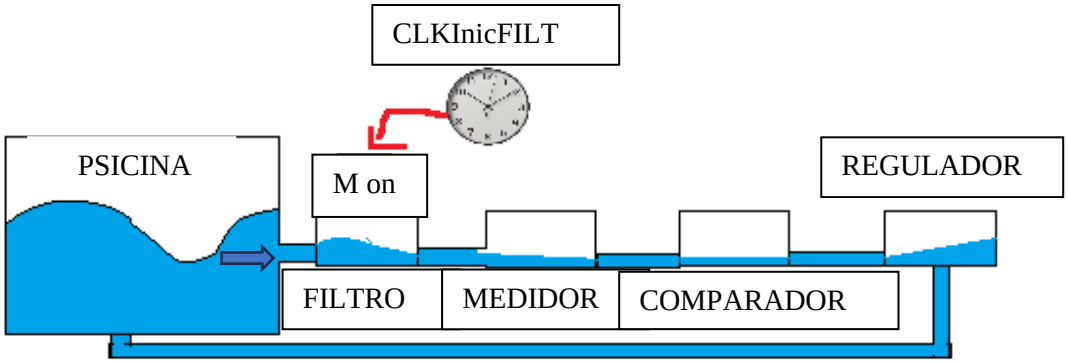
| POUTCTR | Mon | | | | PhS | | PhA | PhB |
|---|---|---|---|---|---|---|---|---|

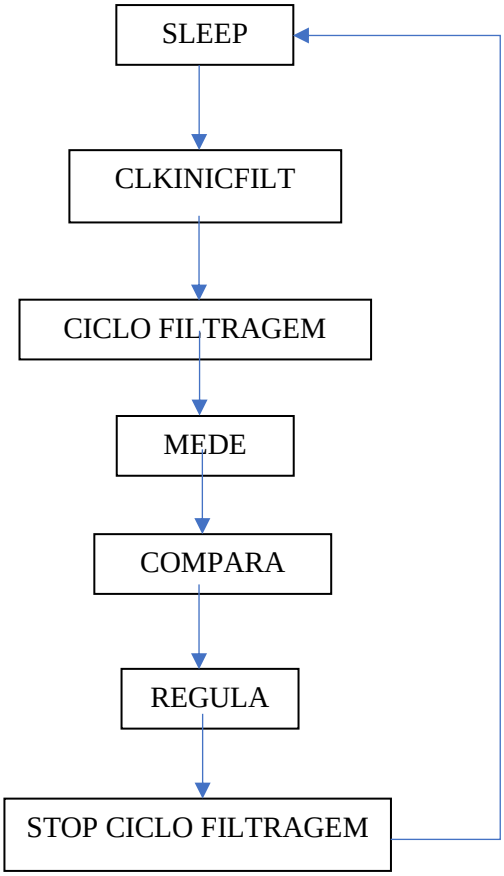
2) Um relógio (ClkInicFilt) ativa o sistema de filtragem de água.
O sistema deve estar num estado de sleep, e quando o relógio sinaliza o inicio de filtragem ativa uma interrupção.
2.1) O relógio (ClkInicFilt) também coloca a 0 lógico o bit menos significativo do porto de entrada PINFILTR.
2.2) Durante o ciclo de Filtragem o sistema deve colocar o bit mais significativo de POUTCTR, Mon a 1.

3) Quando o ciclo de filtragem termina, o relógio ClkTmpFilt fica a 1 lógico e é lido no bit mais significativo do porto PINFILTR.
Nesse caso o sistema de filtragem deve terminar a operação. O bit menos significativo do porto de entrada PINFILTR fica a 1 lógico. Mon deve ser colocado a 0 lógico.

4) Durante o ciclo de filtragem o sistema também deve fazer a regulação do nível de Ph.
4.1) Assim, o sistema tem um sensor que permite ler o nível de Ph.
4.2) Uma leitura do nível de Ph é iniciada colocando um pino PhS a nível lógico 1.
4.3) Depois de ativar PhS o sistema gera uma interrupção quando a leitura do sensor estiver disponível.
4.4) O valor lido no sensor fica disponível no porto de 8 bits PINPH.
4.5) O valor lido deve ser comparado com o valor de referência disponível no porto de 8 bits PINREF.
4.6) Caso o valor lido no sensor seja maior do que o lido no PINREF +3 o PhA que permite diminuir o Ph deve ser colocado a 1.
4.7) Caso o valor lido no sensor seja menor do que o lido no PINREF o PhB que permite aumentar o Ph deve ser colocado a 1.
4.8) Quando o sistema de filtragem for desligado deve terminar o processo de regulação do Ph que deve ser feito de forma cíclica durante o processo de filtragem.
Faça um programa no assembly do arduino para controlar o sistema descrito.
As posições de memória RAM disponíveis estão nos endereços 1000H até 10FFH.




CLKInicFILT
PSICINA
M on
REGULADOR
FILTRO
MEDIDOR
COMPARADOR


SLEEP
CLKINICFILT
CICLO FILTRAGEM
MEDE
COMPARA
REGULA
STOP CICLO FILTRAGEM

# 4. Código em Assembly

```
.include "m2560def.inc"

; Salta para o início do programa e para a rotina de interrupção RINT0
jmp START
jmp RINT0

; Definição de constantes
.equ POUTCTR = PINA
.equ PINREF = PINC
.equ PINph = ADCH

; Vetor de interrupção para o ADC
.org 0x003A
JMP LERADC

START:
    ; Inicialização da stack
    ldi r16, LOW(RAMEND) ; Carrega o byte baixo do endereço final da RAM em r16
    out SPL, r16         ; Define o byte baixo do ponteiro da stack
    ldi r16, HIGH(RAMEND) ; Carrega o byte alto do endereço final da RAM em r16
    out SPH, r16         ; Define o byte alto do ponteiro da stack

    ; Inicialização do modo sleep
    ldi R16, $01         ; Configura o registo de controle do modo sleep
    out SMCR, R16        ; Habilita o modo sleep

    ; Inicialização das interrupções externas
    ldi R16, 0x00        ; Configura o registo de controlo da MCU
    out MCUCR, R16
    ldi R16, 0x03        ; Configura o registo de controlo das interrupções ext.
    sts EICRA, R16
    ldi R16, 0x01        ; Habilita a interrupção externa INT0
    out EIMSK, R16
    sei                  ; Habilita interrupções globais


    ; Configuração das direções dos pinos: 0 = entrada, 1 = saída

    ldi R16, $B3         ; Configura os pinos de PORTA
    out DDRA, R16        ; PORTA: |0|0|0|PHs|0|PhA|PHB|

    ldi R16, $0F         ; Configura os pinos de PORTB
    out DDRB, R16        ; PORTB: (00001111)

    ldi R16, $F0         ; Configura os pinos de PORTC
    out DDRC, R16        ; PORTC: (11110000)
    out DDRD, R16        ; PORTD: Interruptor INT0


    ; Inicialização do ADC
    ldi R16, $20         ; Configura o registo ADMUX
    sts ADMUX, R16
```

```
ModoSleep:
    SLEEP                   ; Entra no modo sleep


CLKInicFilt:
    in R19, PINFILTER       ; Lê o valor do filtro de entrada
    andi R19, $00           ; Coloca a 0 R19
    out POUTCTR, R19        ; Envia o valor para o POUTCTR
    jmp INIC_CICLO_DE_FILTRAGEM ; Pula para INIC_CICLO_DE_FILTRAGEM

INIC_CICLO_DE_FILTRAGEM:
    sbr R18, $80            ; Define Mon=1
    out POUTCTR, R18        ; Envia o valor para o POUTCTR
    jmp MEDIRPH             ; Pula para MEDIRPH

MEDIRPH:
    in R18, PINA            ; Lê o valor de PINA
    andi R18, $88           ; Verifica PhS
    out PORTA, R18          ; Envia o valor para PORTA
    sbrs R18, 3             ; Salta a próxima instrução se o bit 3 de R18 for 0
    rjmp MEDIRPH            ; Repete a leitura se PhS não estiver definido
    in R18, PINA            ; Lê novamente o valor de PINA
    andi R18, $0C           ; Verifica o ADC
    breq MEDIRPH            ; Repete a leitura se a condição for atendida
    jmp STARTADC            ; Pula para STARTADC

STARTADC:
    ldi R16, $CF            ; Inicia a conversão ADC
    sts ADCSRA, R16
    sleep                   ; Entra no modo sleep

loop:
    lds R19, ADCH           ; Lê o valor do ADC
    mov R30, R19            ; Move o valor de R19 para R30
    lsr R19                 ; Desloca R19 quatro vezes para a direita
    lsr R19                 ; Para visualizar os LEDS nas posições mais significc.
    lsr R19
    out PORTC, R19          ; Envia o valor deslocado para PORTC
    ldi R16, $07            ; Reinicia o ADC
    sts ADCSRA, R16

Comparacao:
    in R20, PINREF          ; Lê o valor de PINREF
    ldi R22, $03            ; Carrega o valor 3 em R22
    adc R20, R22            ; Soma R20 com R22 mais o valor de carry
    out PINREF, R20         ; Envia o valor para PINREF
    cp R19, R20             ; Compara R19 com R20
    breq sbtracao           ; Salta para sbtracao se igual
    brpl MAIOR              ; Salta para MAIOR se R19 > R20

    subi R20, $03           ; Subtrai 3 de R20
    out PINREF, R20         ; Envia o valor para PINREF
    cp R19, R20             ; Compara R19 com R20
    brmi MENOR              ; Salta para MENOR se R19 < R20
    jmp N0                  ; Salta para N0

sbtracao:
    subi R20, $03           ; Subtrai 3 de R20
    out PINREF, R20         ; Envia o valor para PINREF
    cp R19, R20             ; Compara R19 com R20
    brmi MENOR              ; Salta para MENOR se R19 < R20
    jmp N0                  ; Salta para N0
```

```
N0:
    cbr R18, $03          ; Limpa os bits 0 e 1 de R18
    sbr R18, $80          ; Define o bit 7 de R18
    out PORTA, R18        ; Envia o valor para PORTA
    jmp Verif             ; Salta para Verif

Verif:
    ; Verifica se deve iniciar o fecho do sistema de filtragem
    in R20, PINA          ; Lê o valor de PINA
    sbrs R20, 4           ; Salta a próxima instrução se o bit 4 de R20 for 0
    jmp STARTADC          ; Pula para STARTADC
    jmp CLKTmpFilt        ; Pula para CLKTmpFilt

MAIOR:
    ldi R18, $8E          ; Define PhA=1
    out PORTA, R18        ; Envia o valor para PORTA
    jmp Verif             ; Salta para Verif

MENOR:
    ldi R18, $8D          ; Define PhB=1
    out PORTA, R18        ; Envia o valor para PORTA
    jmp Verif             ; Salta para Verif

CLKTmpFilt:
; Configura os bits de PORTA
    sbr R18, $70          ; Define bits 4 a 6 de R18
    cbr R18, $80          ; Limpa o bit 7 de R18
    out PORTA, R17        ; Envia o valor para PORTA |0|0|0|0|0|0|0|

RINT0:
    reti                  ; Retorna da interrupção

LERADC:
    reti                  ; Retorna da interrupção
```

# 4. TABELA DE ESTADOS

| PH(adc) | PHref | phA | phB | ESTADO | AÇÃO | VALIDO |
|---|---|---|---|---|---|---|
| 8 | 5(+3)=8 | 0 | 0 | PH IGUAL | MANTEM PH | SIM |
| 0 | 1(+3)=4 | 0 | 1 | PH BAIXO | AUMENTA PH | SIM |
| 10 | 6(+3)=9 | 1 | 0 | PH ALTO | BAIXA PH | SIM |
| 6 | 10(+3)=13 | 0 | 1 | PH BAIXO | AUMENTA PH | SIM |

Exemplo mínimo de valores testados

# 5. **CONCLUSÃO**

Com a realização deste trabalho laboratorial, foi possível adquirir uma valiosa experiência em programação na linguagem Assembly, mais especificamente a utilizada para programar o Arduino ATmega2560. Embora tenha enfrentado diversas dificuldades, e por fim tenha recorrido ao código disponibilizado pelos meus colegas para concluir este projeto, o mesmo permitiu-me adquirir conhecimento sobre o funcionamento dos portos de entrada (PIN) e de saída (PORT) através da simulação e utilizando o software AVR Studio. Apreender e aplicar conceitos explorados nas aulas, como a utilização de interrupções, a aplicação de rotinas, loops, colocar o sistema em modo sleep, técnicas de polling e aprofundar conhecimentos específicos sobre o modo de funcionamento do ADC na recolha de valores de tensão de modo instantâneo. Foram abordados diversos temas de maneira prática, proporcionando um entendimento mais profundo do seu funcionamento.

A utilização do Arduino ATmega2560 implicou um estudo detalhado do seu datasheet, revelando muitas das funcionalidades avançadas que este microcontrolador oferece. Este conhecimento aprofundado sobre o hardware e as suas capacidades foi fundamental para o sucesso da implementação do sistema de controlo, filtragem e regulação do pH da piscina.

Além disso, a necessidade de desenvolver uma estratégia de programação robusta e eficiente proporcionou-me uma experiência valiosa na criação de soluções para problemas específicos. Este processo de planeamento e implementação aprimorou as minhas habilidades em programação, o que certamente facilitará a resolução de desafios futuros em projetos de sistemas embebidos.

O projeto não apenas cumpriu os seus objetivos de automatizar o processo de filtragem e regulação do pH, como também consolidou conhecimentos teóricos através de aplicações práticas. A experiência adquirida com este trabalho contribui significativamente para o desenvolvimento de competências em programação e em engenharia de controlo, preparando-me e incentivando-me a resolver melhor os problemas em projetos futuros.